Experiment 4

Aim: Hands on Solidity Programming Assignments for creating Smart Contracts.

Theory:
Primitive Data Types, Variables, Functions – pure and view

In Solidity, primitive data types are the basic building blocks used when writing smart contracts. Frequently used types include:

- **uint / int**: Integer types used for numerical values. uint represents unsigned numbers while int allows both positive and negative numbers. They come in different sizes such as uint256 or int128.
- **bool**: Used to store logical values like true or false.
- **address**: Stores a 20-byte Ethereum account identifier. It is typically used to represent users or contract addresses.
- **bytes / string**: bytes stores raw binary data, while string is used for textual information.

Variables in Solidity are categorized based on where they are stored:

- **State variables** – saved permanently on the blockchain and accessible throughout the contract.
- **Local variables** – temporary variables created inside functions and discarded after execution.
- **Global variables** – predefined variables supplied by the Ethereum environment, such as msg.sender, msg.value, and block.timestamp.

Functions define the operations performed by a contract. Solidity provides special function types:

- **pure functions**: do not access or modify blockchain data; they only perform calculations using provided inputs.
- **view functions**: can read contract state variables but are not allowed to modify them.

These classifications help ensure predictable behavior and reduce gas costs.

Inputs and Outputs to Functions

Functions in Solidity can receive parameters and return values after executing certain logic. Input parameters allow users or other contracts to supply necessary data to a function. Output values allow the function to return results once processing is complete.

For instance, a function may take a payment amount as input and return a confirmation indicating whether the transaction succeeded. Solidity also supports **named return variables**, which make the function output clearer and easier to understand when reading the code.

Visibility, Modifiers, and Constructors

Function visibility determines who is allowed to call a function:

- **public** – can be called both from within the contract and from external accounts.
- **private** – accessible only inside the contract where it is defined.
- **internal** – accessible within the contract and any contracts that inherit from it.
- **external** – callable only from outside the contract.

Modifiers are reusable pieces of code that alter how functions behave. They are often used for implementing restrictions such as allowing only the contract owner to execute certain actions (onlyOwner).

A **constructor** is a special function that runs once when the contract is deployed. It is mainly used to set initial values, for example assigning the deployer of the contract as the owner.

Control Flow: if-else and Loops

Control flow statements determine how the program executes based on conditions.

- **if-else statements** enable decision making in smart contracts. For example, they can verify whether an account has sufficient balance before allowing a payment.
- **Loops** such as for, while, and do-while allow repetitive execution of code blocks, like processing elements in a list.

However, loops should be used cautiously in Solidity because each iteration consumes gas, and excessive iterations can make transactions very costly.

Data Structures: Arrays, Mappings, Structs, Enums

Solidity provides several data structures for organizing information efficiently.

- **Arrays** store collections of elements in sequence. They can have a fixed length or grow dynamically.
- **Mappings** are key-value storage structures used for quick data retrieval. A common example is storing balances with mapping(address => uint). Unlike arrays, mappings cannot be iterated over directly.
- **Structs** allow developers to combine multiple related variables into a single custom data type, such as grouping a player's name and score together.

- **Enums** define a limited set of possible constant values, improving code clarity. For example, a process state could be represented as Pending, Active, or Closed.

Data Locations

Solidity specifies where variables are stored using **data locations**:

- **storage** – permanent storage on the blockchain. State variables are stored here.
- **memory** – temporary storage used during function execution. Data disappears once the function finishes.
- **calldata** – a read-only location used mainly for parameters passed to external functions. It is cheaper in terms of gas compared to memory.

Choosing the appropriate data location helps improve performance and reduce execution costs.

Transactions: Ether and Wei, Gas and Gas Price, Sending Transactions

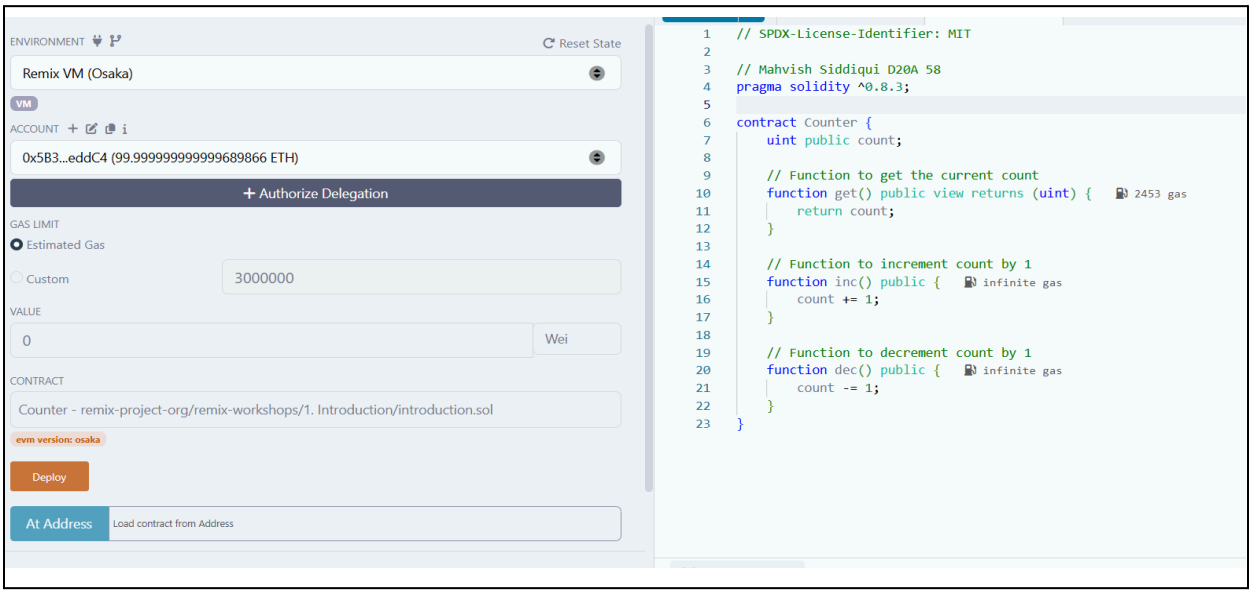
Ether and Wei:
Ether is the main currency used in the Ethereum network, but internally all transactions are calculated in **Wei**, the smallest denomination (1 Ether equals (10^{18}) Wei). This allows very precise financial calculations.

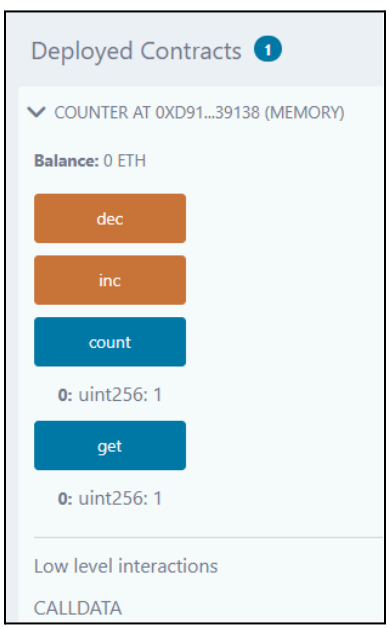
Gas and Gas Price:
Executing operations on Ethereum requires gas, which represents the computational resources needed to run a transaction. The **gas price** specifies how much Ether a user is willing to pay per unit of gas. Transactions offering higher gas prices are typically processed faster by validators.

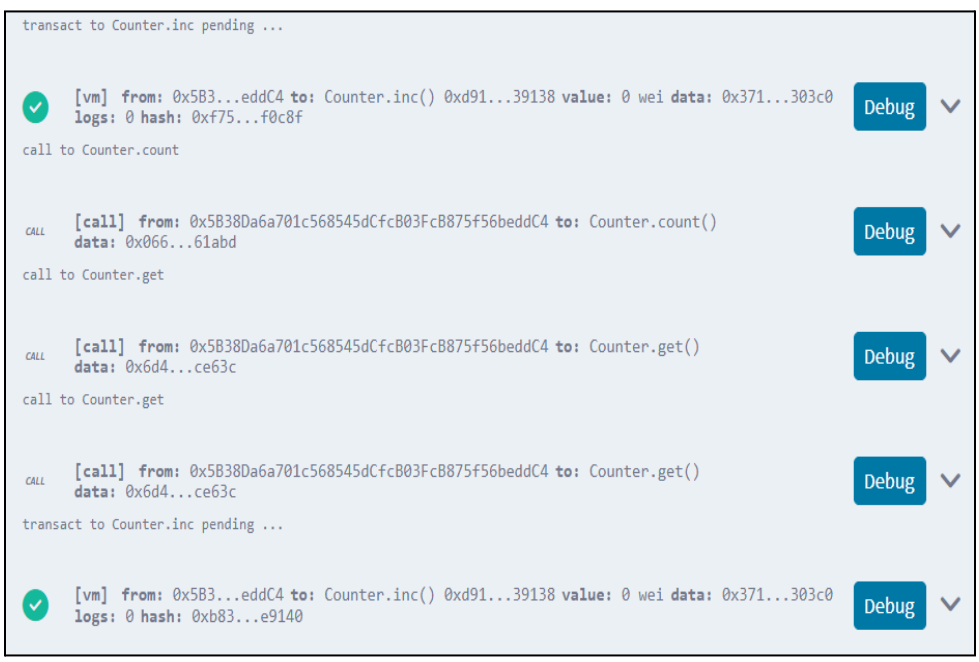
Sending Transactions:
Transactions can transfer Ether between accounts or interact with smart contracts. Solidity provides functions such as transfer() and send() for simple payments, while call() offers a more flexible method for interacting with other contracts. Each transaction consumes gas, making efficiency in smart contract design crucial.

Implementation:

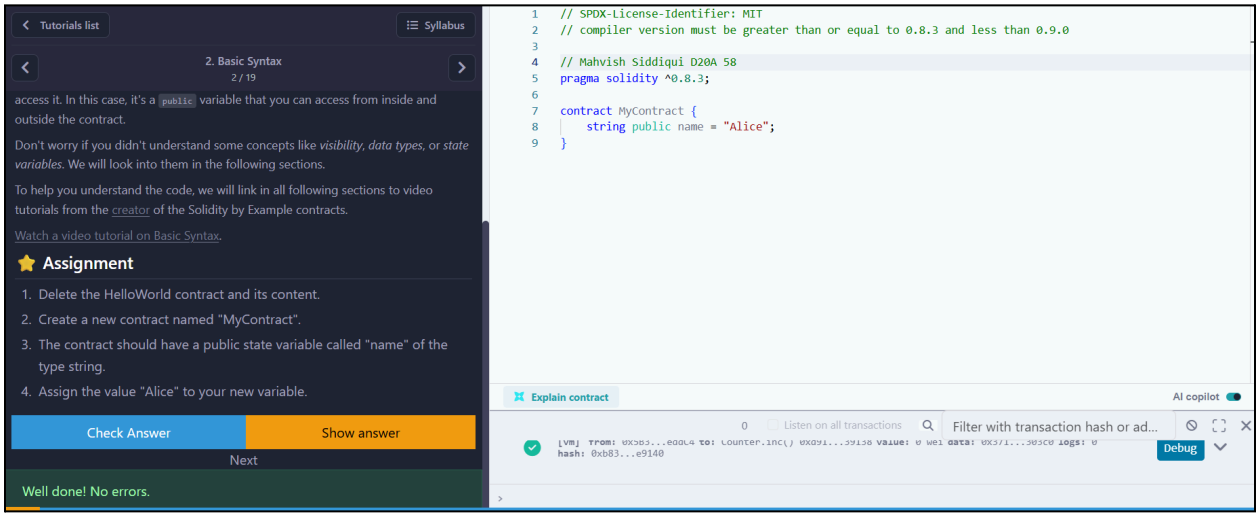
Tutorial 1



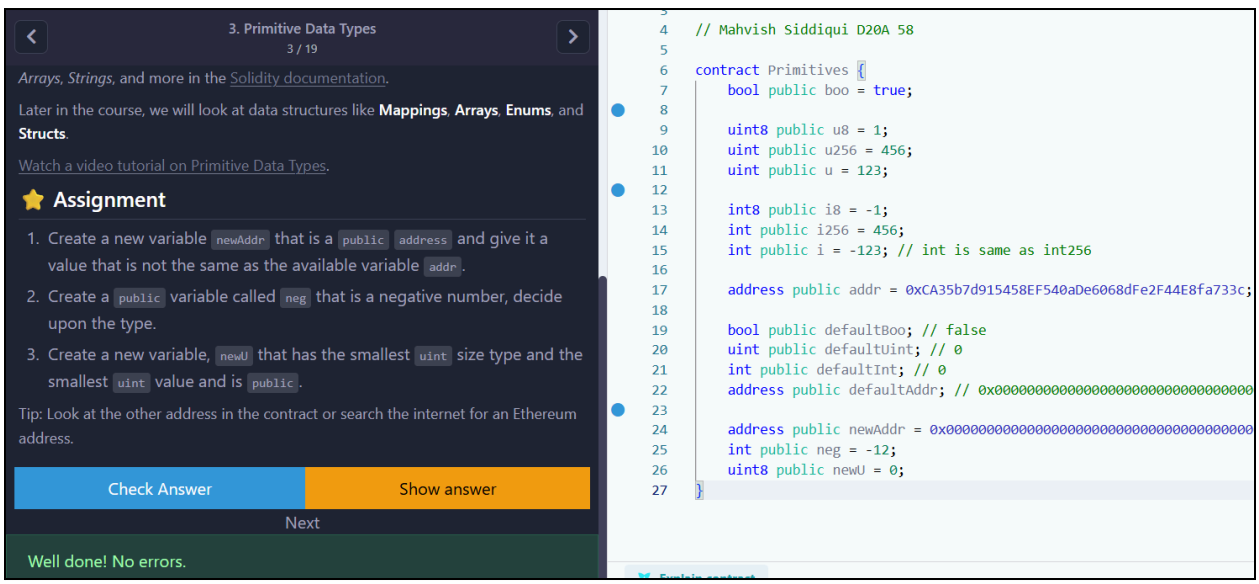




Tutorial 2



Tutorial 3



Tutorial 4

4. Variables
4 / 19

used to retrieve information about the blockchain, particular addresses, contracts, and transactions.

In this example, we use `block.timestamp` (line 14) to get a Unix timestamp of when the current block was generated and `msg.sender` (line 15) to get the caller of the contract function's address.

A list of all Global Variables is available in the Solidity documentation.

Watch video tutorials on State Variables, Local Variables, and Global Variables.

Assignment

1. Create a new public state variable called `blockNumber`.
2. Inside the function `doSomething()`, assign the value of the current block number to the state variable `blockNumber`.

Tip: Look into the global variables section of the Solidity documentation to find out how to read the current block number.

Check Answer | Show answer

Next

Well done! No errors.

```
4
5   // Mahvish Siddiqui D20A 58
6
7   contract Variables {
8       // State variables are stored on the blockchain.
9       string public text = "Hello";
10      uint public num = 123;
11
12      uint public blockNumber;
13
14      function doSomething() public {    22300 gas
15          blockNumber = block.number;
16      }
17  }
18
```

Tutorial 5

5.1 Functions - Reading and Writing to a State Variable
5 / 19

If the function takes inputs like our `set` function (line 9), you must specify the parameter types and names. A common convention is to use an underscore as a prefix for the parameter name to distinguish them from state variables.

You can then set the visibility of a function and declare them `view` or `pure` as we do for the `get` function if they don't modify the state. Our `get` function also returns values, so we have to specify the return types. In this case, it's a `uint` since the state variable `num` that the function returns is a `uint`.

We will explore the particularities of Solidity functions in more detail in the following sections.

Watch a video tutorial on Functions.

Assignment

1. Create a public state variable called `b` that is of type `bool` and initialize it to `true`.
2. Create a public function called `get_b` that returns the value of `b`.

Check Answer | Show answer

Next

Well done! No errors.

```
3
4   // Mahvish Siddiqui D20A 58
5
6   contract SimpleStorage {
7       // State variable to store a number
8       uint public num;
9       bool public b = true;
10
11      // You need to send a transaction to write to a state var
12      function set(uint _num) public {    22536 gas
13          num = _num;
14      }
15
16      // You can read from a state variable without sending a t
17      function get() public view returns (uint) {    2475 gas
18          return num;
19      }
20
21      function get_b() public view returns (bool) {    2539 ga
22          return b;
23      }
24  }
```

Tutorial 6

5.2 Functions - View and Pure
6 / 19

5. Using inline assembly that contains certain opcodes."

From the Solidity documentation.

You can declare a pure function using the keyword `pure`. In this contract, `add` (line 13) is a pure function. This function takes the parameters `i` and `j`, and returns the sum of them. It neither reads nor modifies the state variable `x`.

In Solidity development, you need to optimise your code for saving computation cost (gas cost). Declaring functions view and pure can save gas cost and make the code more readable and easier to maintain. Pure functions don't have any side effects and will always return the same result if you pass the same arguments.

Watch a video tutorial on View and Pure Functions.

Assignment

Create a function called `addToX2` that takes the parameter `y` and updates the state variable `x` with the sum of the parameter and the state variable `x`.

Check Answer | Show answer

Next

Well done! No errors.

```
4   // Mahvish Siddiqui D20A 58
5
6   contract ViewAndPure {
7       uint public x = 1;
8
9       // Promise not to modify the state.
10      function addToX(uint y) public view returns (uint) {    infinite gas
11          return x + y;
12      }
13
14      // Promise not to modify or read from the state.
15      function add(uint i, uint j) public pure returns (uint) {    infinite gas
16          return i + j;
17      }
18
19      function addToX2(uint y) public returns (uint) {    infinite gas
20          x = x + y;
21          return x;
22      }
23  }
```

Explain contract

5.3 Functions - Modifiers and Constructors
7 / 19

A constructor function is executed upon the creation of a contract. You can use it to run contract initialization code. The constructor can have parameters and is especially useful when you don't know certain initialization values before the deployment of the contract.

You declare a constructor using the `constructor` keyword. The constructor in this contract (line 11) sets the initial value of the owner variable upon the creation of the contract.

Watch a video tutorial on Function Modifiers.

Assignment

1. Create a new function, `increaseX` in the contract. The function should take an input parameter of type `uint` and increase the value of the variable `x` by the value of the input parameter.
2. Make sure that x can only be increased.
3. The body of the function `increaseX` should be empty.

Tip: Use modifiers.

Check Answer | Show answer

Next

Well done! No errors.

```
45          locked = false;
46      }
47
48      function decrement(uint i) public noReentrancy {    infinite gas
49          x -= i;
50
51          if (i > 1) {
52              decrement(i - 1);
53          }
54      }
55
56      modifier biggerThan0(uint y) {
57          require(y > 0, "Not bigger than x");
58          _;
59      }
60
61      modifier increaseXbyY(uint y) {
62          _;
63          x = x + y;
64      }
65
66      function increaseX(uint y) public onlyOwner biggerThan0(y) increaseXbyY(y){    infinite ga
67      }
68  }
69
70  // Mahvish Siddiqui D20A 58
```

5.4 Functions - Inputs and Outputs
8 / 19

There are a few restrictions and best practices for the input and output parameters of contract functions.

"*[Mappings] cannot be used as parameters or return parameters of contract functions that are publicly visible.*" From the Solidity documentation.

Arrays can be used as parameters, as shown in the function `arrayInput` (line 71). Arrays can also be used as return parameters as shown in the function `arrayOutput` (line 76).

You have to be cautious with arrays of arbitrary size because of their gas consumption. While a function using very large arrays as inputs might fail when the gas costs are too high, a function using a smaller array might still be able to execute.

Watch a video tutorial on Function Outputs.

Assignment

Create a new function called `returnTwo` that returns the values `-2` and `true` without using a return statement.

Check Answer | Show answer

Next

Well done! No errors.

```
74      uint[] public arr;
75
76      function arrayOutput() public view returns (uint[] memory) {
77          return arr;
78      }
79
80
81      function returnTwo() public pure    472 gas
82          returns (
83              int x,
84              bool b
85          )
86      {
87          x = -2;
88          b = true;
89      }
90  }
91
92  // Mahvish Siddiqui D20A 58
```

Tutorial 9

6. Visibility
9 / 19

- Can be called from other contracts or transactions
- State variables can not be `external`

In this example, we have two contracts, the `Base` contract (line 4) and the `Child` contract (line 55) which inherits the functions and state variables from the `Base` contract.

When you uncomment the `testPrivateFunc` (lines 58-60) you get an error because the child contract doesn't have access to the private function `privateFunc` from the `Base` contract.

If you compile and deploy the two contracts, you will not be able to call the functions `privateFunc` and `internalFunc` directly. You will only be able to call them via `testPrivateFunc` and `testInternalFunc`.

Watch a video tutorial on Visibility.

Assignment

Create a new function in the `Child` contract called `testInternalVar` that returns the values of all state variables from the `Base` contract that are possible to return.

Check Answer | Show answer

Next

Well done! No errors.

```
49      string internal internalVar = "my internal variable";
50      string public publicVar = "my public variable";
51      // State variables cannot be external so this code won't compile.
52      // string external externalVar = "my external variable";
53  }
54
55  contract Child is Base {
56      // Inherited contracts do not have access to private functions
57      // and state variables.
58      // function testPrivateFunc() public pure returns (string memory) {
59      //     return privateFunc();
60      // }
61
62      // Internal function call be called inside child contracts.
63      function testInternalFunc() public pure override returns (string memory) {
64          return internalFunc();
65      }
66
67      function testInternalVar() public view returns (string memory, string memory)
68          return (internalVar, publicVar);
69      }
70  }
71
72  // Mahvish Siddiqui D20A 58
```

Explain contract

Tutorial 10

7.1 Control Flow - If/Else
10 / 19

else if

With the `else if` statement we can combine several conditions.

If the first condition (line 6) of the foo function is not met, but the condition of the `else if` statement (line 8) becomes true, the function returns `1`.

Watch a video tutorial on the If/Else statement.

Assignment

Create a new function called `evenCheck` in the `IfElse` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternery operator to return the result of the `evenCheck` function.

Tip: The modulo (%) operator produces the remainder of an integer division.

Check Answer | Show answer

Next

Well done! No errors.

```
6           if (x < 10) {
7               return 0;
8           } else if (x < 20) {
9               return 1;
10          } else {
11              return 2;
12          }
13      }
14
15      function ternary(uint _x) public pure returns (uint) {    infinite gas
16          // if (_x < 10) {
17          //     return 1;
18          // }
19          // return 2;
20
21          // shorthand way to write if / else statement
22          return _x < 10 ? 1 : 2;
23      }
24
25      function evenCheck(uint y) public pure returns (bool) {    infinite gas
26          return y%2 == 0 ? true : false;
27      }
28  }
29
30  // Mahvish Siddiqui D20A 58
```

Explain contract

Tutorial 11

7.2 Control Flow - Loops
11 / 19

continue

The `continue` statement is used to skip the remaining code block and start the next iteration of the loop. In this contract, the `continue` statement (line 10) will prevent the second if statement (line 12) from being executed.

break

The `break` statement is used to exit a loop. In this contract, the break statement (line 14) will cause the for loop to be terminated after the sixth iteration.

Watch a video tutorial on Loop statements.

Assignment

1. Create a public `uint` state variable called count in the `Loop` contract.
2. At the end of the for loop, increment the count variable by 1.
3. Try to get the count variable to be equal to 9, but make sure you don't edit the `break` statement.

Check Answer | Show answer

Next

Well done! No errors.

```
4   contract Loop {
5       uint public count;
6       function loop() public{    infinite gas
7           // for loop
8           for (uint i = 0; i < 10; i++) {
9               if (i == 5) {
10                  // Skip to next iteration with continue
11                  continue;
12              }
13              if (i == 5) {
14                  // Exit loop with break
15                  break;
16              }
17              count++;
18          }
19
20          uint j;
21          while (j < 10) {
22              j++;
23          }
24      }
25  }
26
27  // Mahvish Siddiqui D20A 58
```

Tutorial 12

8.1 Data Structures - Arrays
12 / 19

(line 42). When we remove an element with the `delete` operator all other elements stay the same, which means that the length of the array will stay the same. This will create a gap in our array. If the order of the array is not important, then we can move the last element of the array to the place of the deleted element (line 46), or use a mapping. A mapping might be a better choice if we plan to remove elements in our data structure.

Array length

Using the length member, we can read the number of elements that are stored in an array (line 35).

Watch a video tutorial on Arrays.

Assignment

1. Initialize a public fixed-sized array called `arr3` with the values 0, 1, 2. Make the size as small as possible.
2. Change the `getArr()` function to return the value of `arr3`.

Check Answer | Show answer

Next

Well done! No errors.

```
47  contract CompactArray {
48      uint[] public arr;
49
50      // Deleting an element creates a gap in the array.
51      // One trick to keep the array compact is to
52      // move the last element into the place to delete.
53      function remove(uint index) public {    infinite gas
54          // Move the last element into the place to delete
55          arr[index] = arr[arr.length - 1];
56          // Remove the last element
57          arr.pop();
58      }
59
60      function test() public {    infinite gas
61          arr.push(1);
62          arr.push(2);
63          arr.push(3);
64          arr.push(4);
65
66          remove(1);
67          remove(2);
68      }
69  }
70
71  // Mahvish Siddiqui D20A 58
```

8.2 Data Structures - Mappings

13 / 19

We set a new value for a key by providing the mapping's name and key in brackets and assigning it a new value (line 16).

Removing values

We can use the delete operator to delete a value associated with a key, which will set it to the default value of 0. As we have seen in the arrays section.

Watch a video tutorial on Mappings.

Assignment

1. Create a public mapping `balances` that associates the key type `address` with the value type `uint`.
2. Change the functions `get` and `remove` to work with the mapping balances.
3. Change the function `set` to create a new entry to the balances mapping, where the key is the address of the parameter and the value is the balance associated with the address of the parameter.

Check Answer | Show answer

Next

Well done! No errors.

```
25  contract NestedMapping {
26      // Nested mapping (mapping from address to another mapping)
27      mapping(address => mapping(uint => bool)) public nested;
28
29      function get(address _addr1, uint _i) public view returns (bool) {
30          // You can get values from a nested mapping
31          // even when it is not initialized
32          return nested[_addr1][_i];
33      }
34
35      function set(    25199 gas
36          address _addr1,
37          uint _i,
38          bool _boo
39      ) public {
40          nested[_addr1][_i] = _boo;
41      }
42
43      function remove(address _addr1, uint _i) public {    25045 gas
44          delete nested[_addr1][_i];
45      }
46  }
47
48  // Mahvish Siddiqui D20A 58
```

Explain contract

8.3 Data Structures - Structs

14 / 19

Key-value mapping: We provide the name of the struct and the keys and values as a mapping inside curly braces (line 19).

Initialize and update a struct: We initialize an empty struct first and then update its member by assigning it a new value (line 23).

Accessing structs

To access a member of a struct we can use the dot operator (line 33).

Updating structs

To update a structs' member we also use the dot operator and assign it a new value (lines 39 and 45).

Watch a video tutorial on Structs.

Assignment

Create a function `remove` that takes a `uint` as a parameter and deletes a struct member with the given index in the `todos` mapping.

Check Answer | Show answer

Next

Well done! No errors.

```
33          return (todo.text, todo.completed);
34      }
35
36      // update text
37      function update(uint _index, string memory _text) public {
38          Todo storage todo = todos[_index];
39          todo.text = _text;
40      }
41
42      function toggleCompleted(uint _index) public {    28995 gas
43          Todo storage todo = todos[_index];
44          todo.completed = !todo.completed;
45      }
46
47      function remove(uint _index) public {    infinite gas
48          delete todos[_index];
49      }
50  }
51
52  // Mahvish Siddiqui D20A 58
```

Explain contract

Tutorial 15

8.4 Data Structures - Enums

15 / 19

We can update the enum value of a variable by assigning it the `uint` representing the enum member (line 30). Shipped would be 1 in this example. Another way to update the value is using the dot operator by providing the name of the enum and its member (line 35).

Removing an enum value

We can use the delete operator to delete the enum value of the variable, which means as for arrays and mappings, to set the default value to 0.

Watch a video tutorial on Enums.

Assignment

1. Define an enum type called `Size` with the members `S`, `M`, and `L`.
2. Initialize the variable `sizes` of the enum type `Size`.
3. Create a getter function `getSize()` that returns the value of the variable `sizes`.

Check Answer | Show answer

Next

Well done! No errors.

```
4   // Mahvish Siddiqui D20A 58
5   contract Enum {
6       // Enum representing shipping status
7       enum Status {
8           Pending,
9           Shipped,
10          Accepted,
11          Rejected,
12          Canceled
13      }
14
15      enum Size {
16          S,
17          M,
18          L
19      }
20
21      // Default value is the first element listed in
22      // definition of the type, in this case "Pending"
23      Status public status;
24      Size public sizes;
25
26      function get() public view returns (Status) {    2605 gas
27          return status;
28      }
29
30      function getSize() public view returns (Size) {    2633 gas
```

Tutorial 16

Tutorials list | Syllabus

9. Data Locations

16 / 19

affect the values stored in the mapping `myStructs` (line 10).

As we said in the beginning, when creating contracts we have to be mindful of gas costs. Therefore, we need to use data locations that require the lowest amount of gas possible.

Assignment

1. Change the value of the `myStruct` member `foo`, inside the `function f`, to 4.
2. Create a new struct `myMemStruct2` with the data location *memory* inside the `function f` and assign it the value of `myMemStruct`. Change the value of the `myMemStruct2` member `foo` to 1.
3. Create a new struct `myMemStruct3` with the data location *memory* inside the `function f` and assign it the value of `myStruct`. Change the value of the `myMemStruct3` member `foo` to 3.
4. Let the function f return `myStruct`, `myMemStruct2`, and `myMemStruct3`.

Tip: Make sure to create the correct return types for the function `f`.

Check Answer | Show answer

Next

Well done! No errors.

```
3   // Mahvish Siddiqui D20A 58
4   contract DataLocations {
5       uint[] public arr;
6       mapping(uint => address) map;
7       struct MyStruct {
8           uint foo;
9       }
10      mapping(uint => MyStruct) public myStructs;
11
12      function f() public returns (MyStruct memory, MyStruct memory, MyStruct memory){
26      }
27
28      function _f(    undefined gas
29          uint[] storage _arr,
30          mapping(uint => address) storage _map,
31          MyStruct storage _myStruct
32      ) internal {...
34      }
35
36      function g(uint[] memory _arr) public returns (uint[] memory) {    infinite gas
37          // do something with memory array
38          arr[0] = 1;
39      }
40
41      function h(uint[] calldata _arr) external {    468 gas
42          // do something with calldata array
43          // _arr[0] = 1;
44      }
45  }
```

Tutorial 17

10.1 Transactions - Ether and Wei

17 / 19

`wei`

Wei is the smallest subunit of *Ether*, named after the cryptographer Wei Dai. *Ether* numbers without a suffix are treated as `wei` (line 7).

`gwei`

One `gwei` (giga-wei) is equal to 1,000,000,000 (10^9) `wei`.

`ether`

One `ether` is equal to 1,000,000,000,000,000,000 (10^18) `wei` (line 11).

Watch a video tutorial on Ether and Wei.

Assignment

1. Create a `public` `uint` called `oneGWei` and set it to 1 `gwei`.
2. Create a `public` `bool` called `isOneGWei` and set it to the result of a comparison operation between 1 gwei and 10^9.

Tip: Look at how this is written for `gwei` and `ether` in the contract.

Check Answer | Show answer

Next

Well done! No errors.

```
4
5   // Mahvish Siddiqui D20A 58
6
7   contract EtherUnits {
8       uint public oneWei = 1 wei;
9       // 1 wei is equal to 1
10      bool public isOneWei = 1 wei == 1;
11
12      uint public oneEther = 1 ether;
13      // 1 ether is equal to 10^18 wei
14      bool public isOneEther = 1 ether == 1e18;
15
16      uint public oneGwei = 1 gwei;
17      bool public isOneGwei = 1 gwei == 1e9;
18  }
```

Tutorial 18

10.2 Transactions - Gas and Gas Price

18 / 19

Gas limit

When sending a transaction, the sender specifies the maximum amount of gas that they are willing to pay for. If they set the limit too low, their transaction can run out of *gas* before being completed, reverting any changes being made. In this case, the *gas* was consumed and can't be refunded.

Learn more about *gas* on ethereum.org.

Watch a video tutorial on Gas and Gas Price.

Assignment

Create a new `public` state variable in the `Gas` contract called `cost` of the type `uint`. Store the value of the gas cost for deploying the contract in the new variable, including the cost for the value you are storing.

Tip: You can check in the Remix terminal the details of a transaction, including the gas cost. You can also use the Remix plugin *Gas Profiler* to check for the gas cost of transactions.

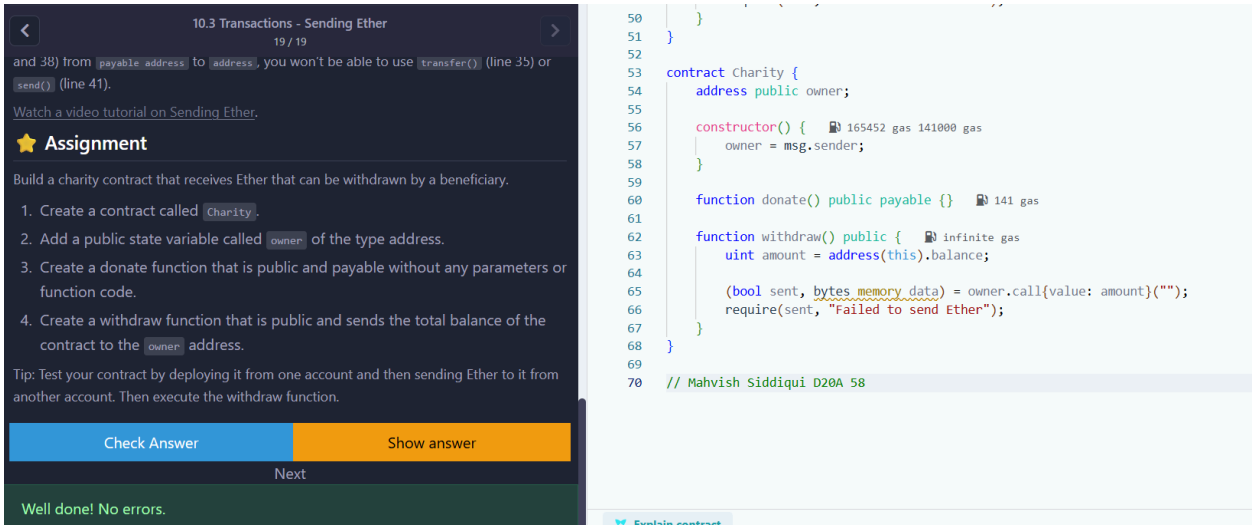
Check Answer | Show answer

Next

Well done! No errors.

```
4   // Mahvish Siddiqui D20A 58
5
6
7   contract Gas {
8       uint public i = 0;
9       uint public cost = 170367;
10
11      // Using up all of the gas that you send causes your transaction to fail.
12      // State changes are undone.
13      // Gas spent are not refunded.
14      function forever() public {    infinite gas
15          // Here we run a loop until all of the gas are spent
16          // and the transaction fails
17          while (true) {
18              i += 1;
19          }
20      }
21  }
```

Tutorial 19



Conclusion:

Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The hands-on practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts. This experiment provided a strong foundation for developing and managing smart contracts efficiently.